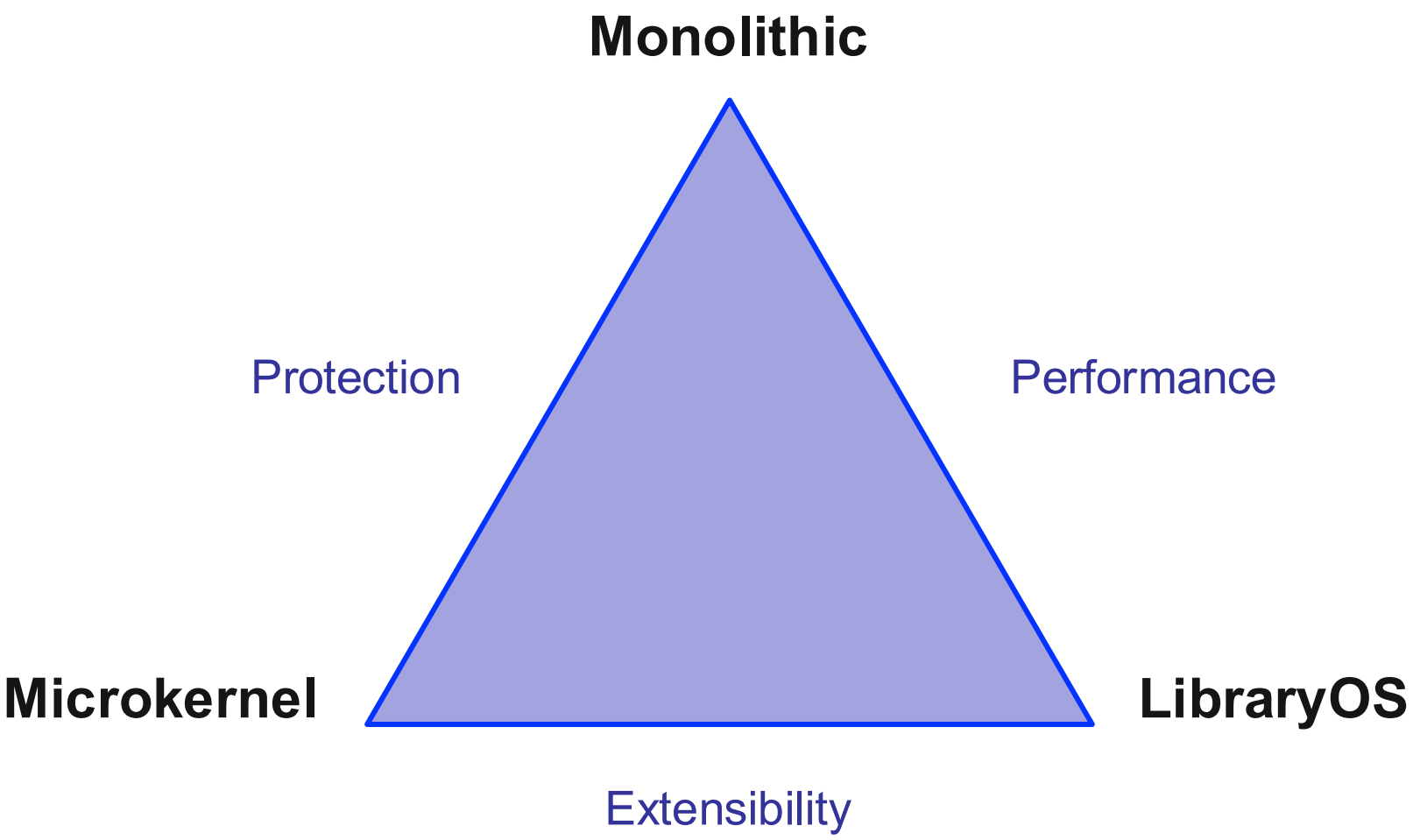
CS 202
Advanced Operating Systems

Winter 26

Lecture 5: Safe Extension of Kernel

Instructor: Chengyu Song

Trade-offs of OS architectures

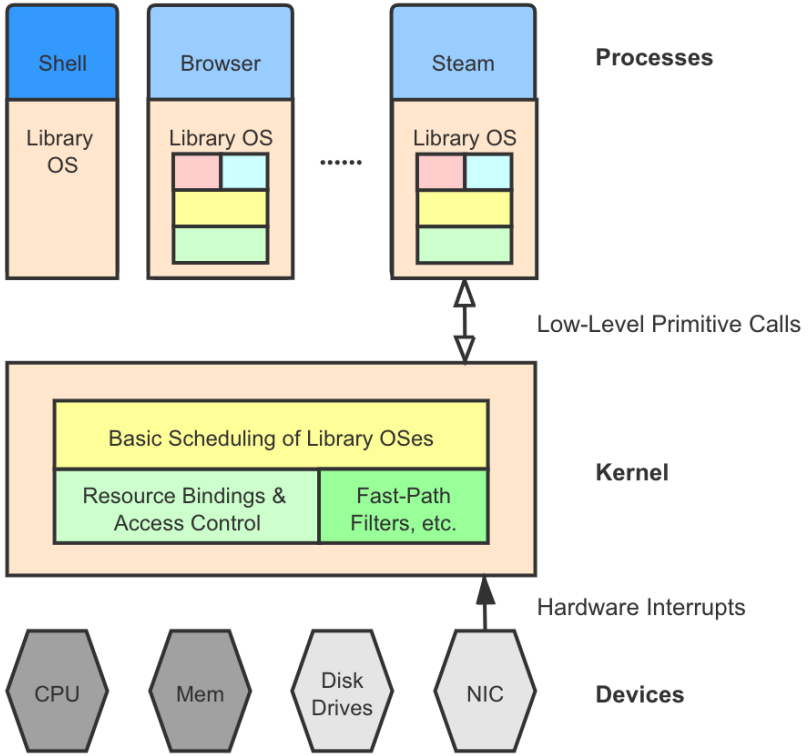


LibOS vs. Virtual Machine

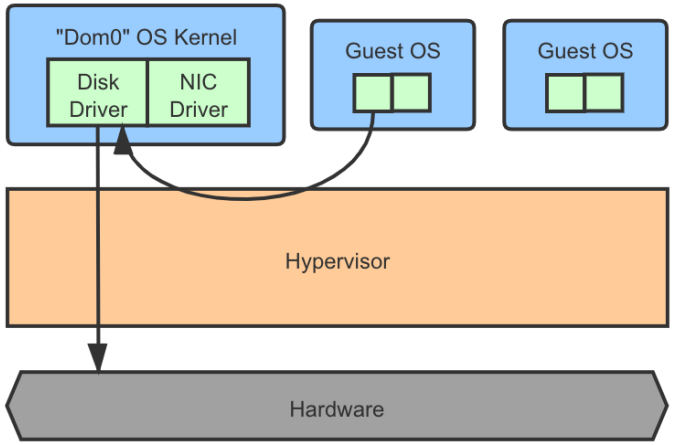
Can we do better today?

Can we vibe code libOS?

- LibraryOS
  - exokernel, graphene



- Hypervisors
  - Xen

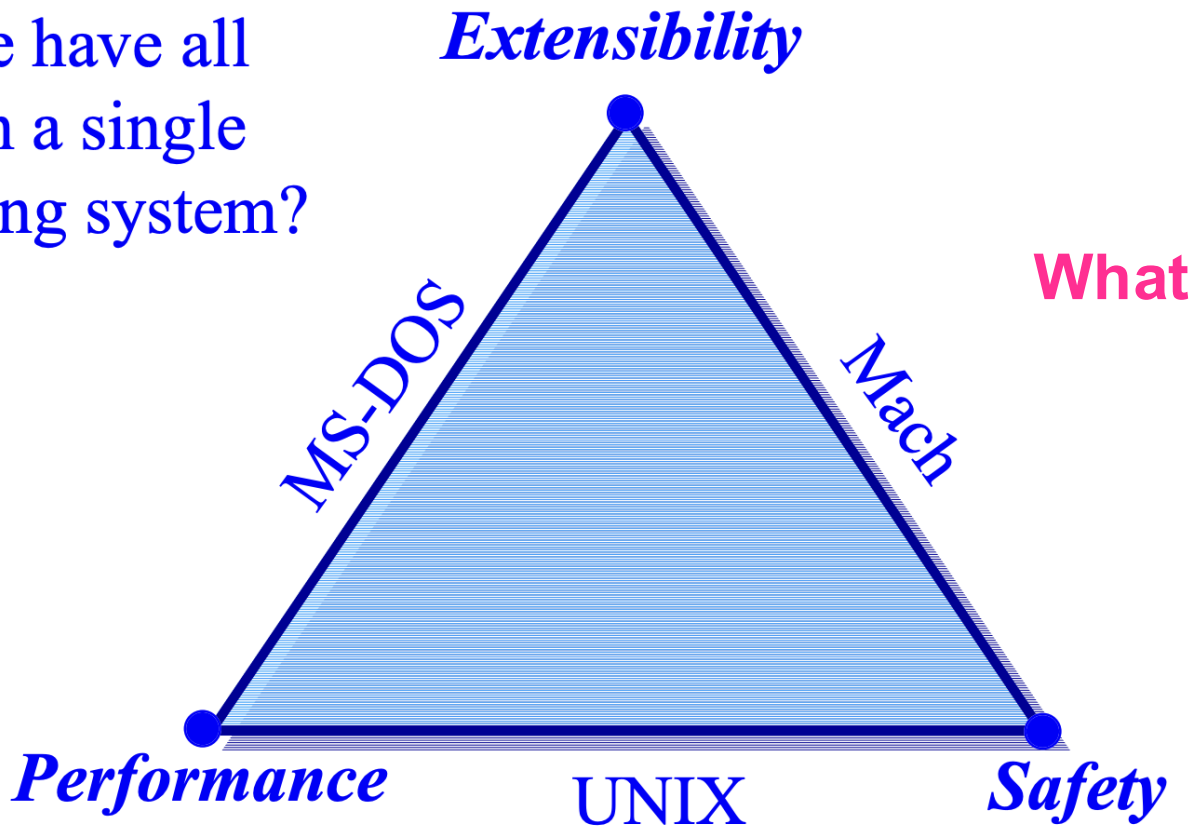


SPIN: safely extend monolithic kernels

- Goals
  - Extensibility
    - » Applications can dynamically extend system to provide specialized services
  - Safety
    - » Kernel is protected from actions of extensions
  - Performance
    - » Extensibility and safety have low cost
- How?
  - Use of **language features** for extensions!

SPIN: Goals

Can we have all three in a single operating system?

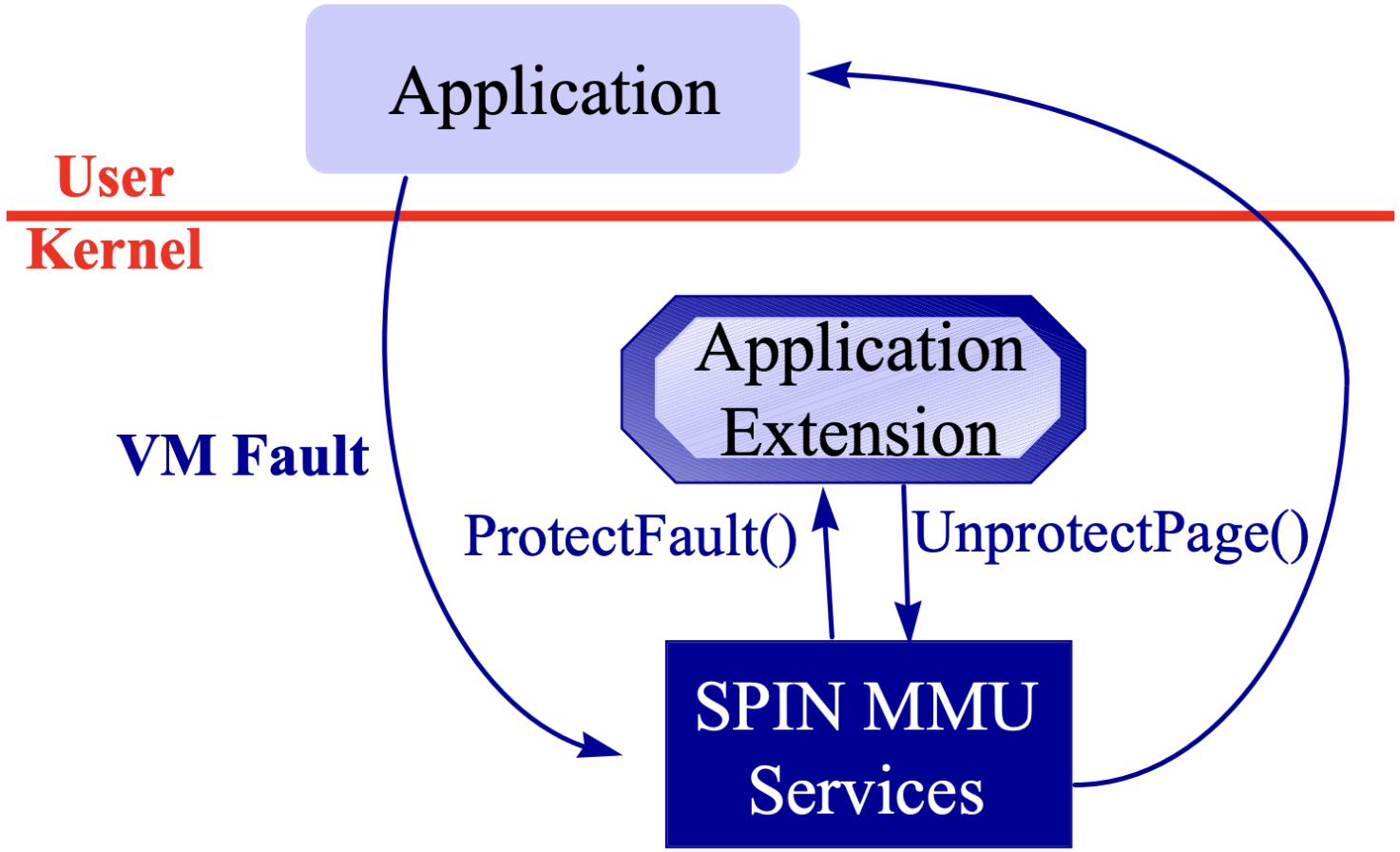


What are the threats to safety?

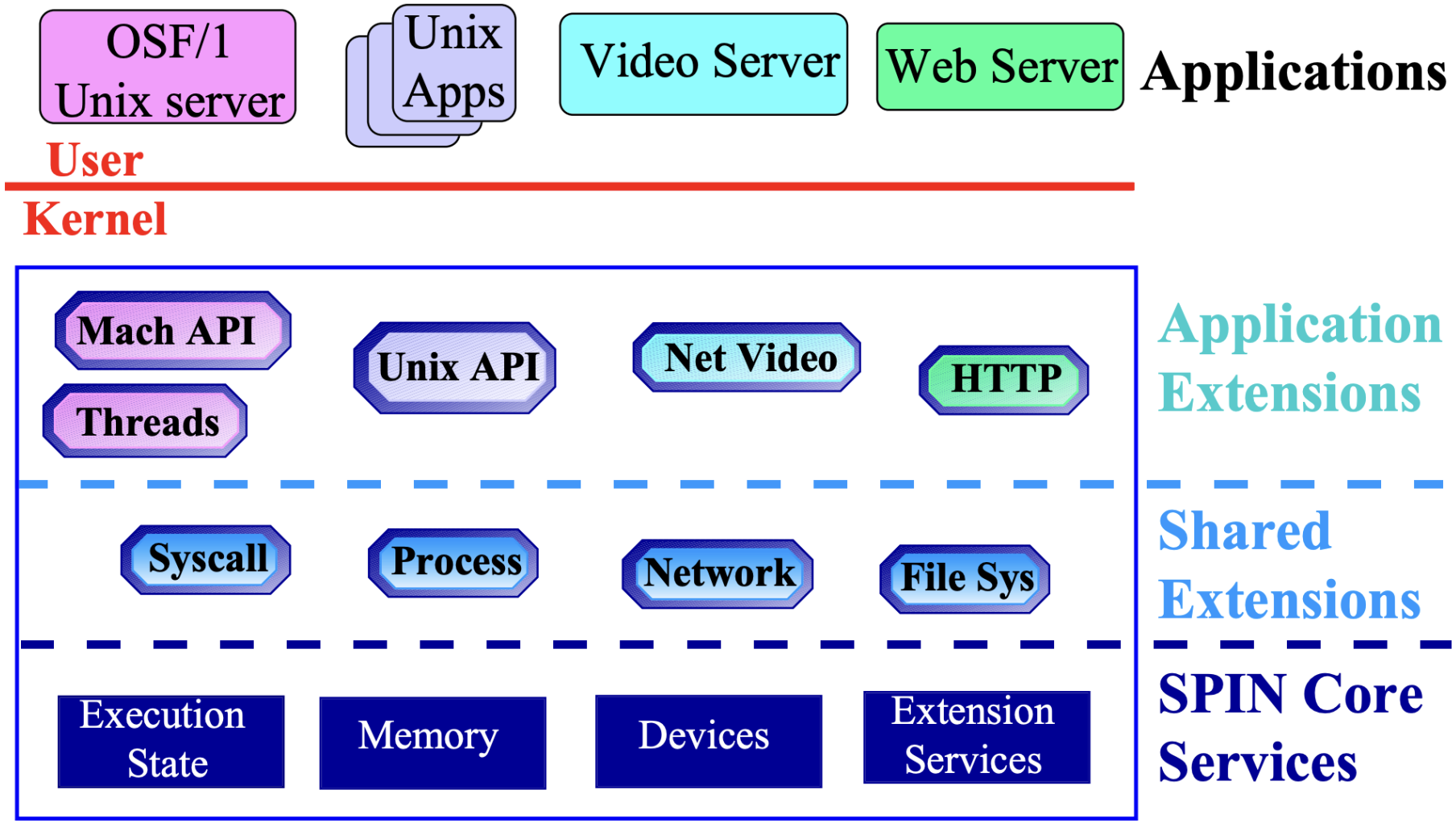
SPIN: approaches

- Put extension code in the kernel
  - Cheap communication
- Use language protection features
  - Static safety
- Dynamically interpose on any service
  - Fine-grained extensibility

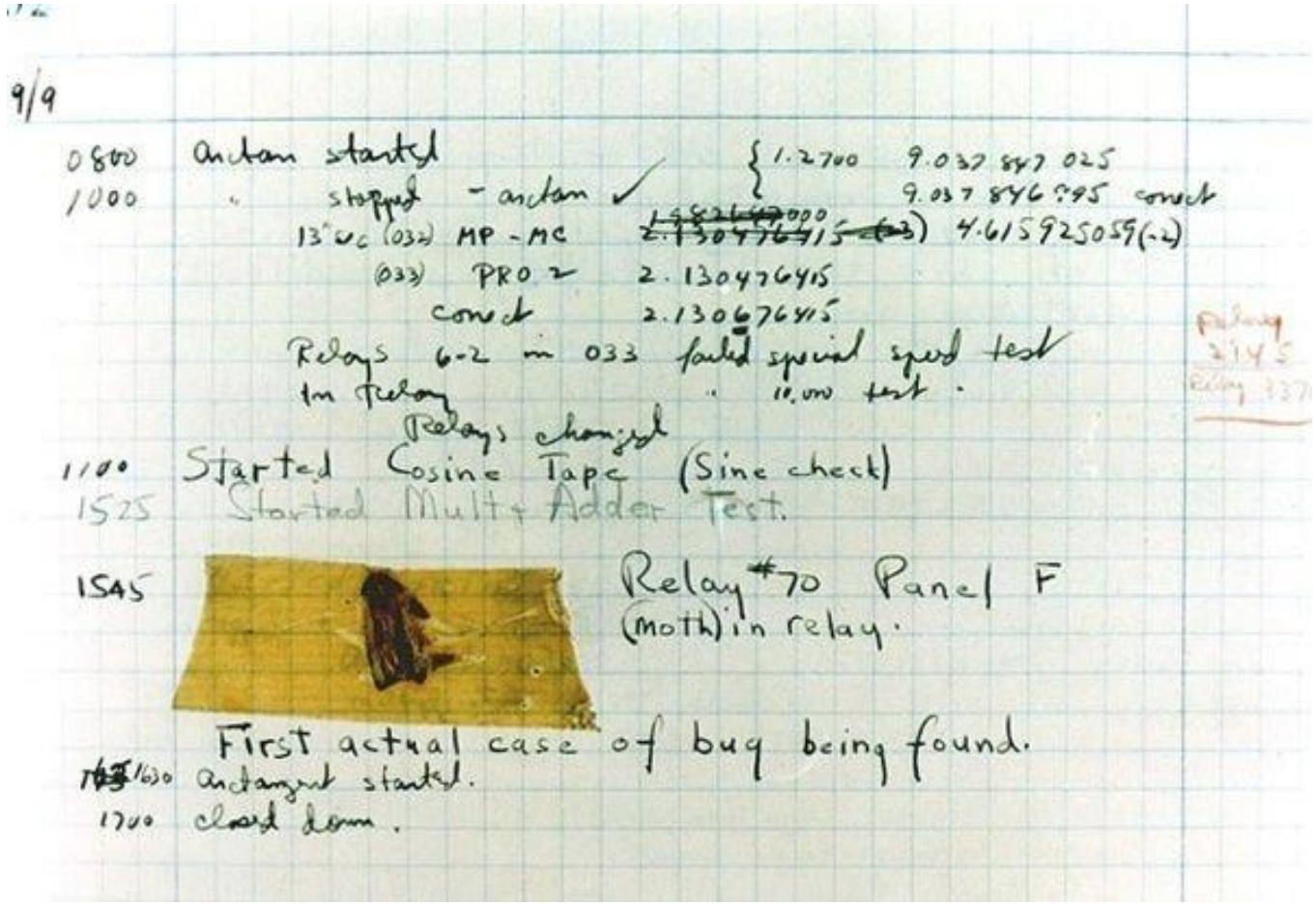
SPIN: how extension looks like



SPIN: architecture



SPIN: threats to safety



SPIN: a language approach

" In the end we shall make thoughtcrime literally impossible, because there will be no words in which to express it. "

SPIN: language-based protection

- Modula-3
  - Memory safe
  - Interfaces for hiding resources
  - Cheap capabilities
- Designed by DEC and Olivetti (1980s)
  - Descendent from Mesa
- Modern language (at that time)
  - Interfaces, type safety, threads, exceptions
- “Died” together with DEC (acquired by Compaq in 1998)

SPIN: performance & safety

- Co-location
  - Extensions dynamically linked into same memory-space as kernel
  - Fast communication
- Enforced modularity
  - Modula 3 extensions provide compile-time protection (memory and privileged instructions protected)

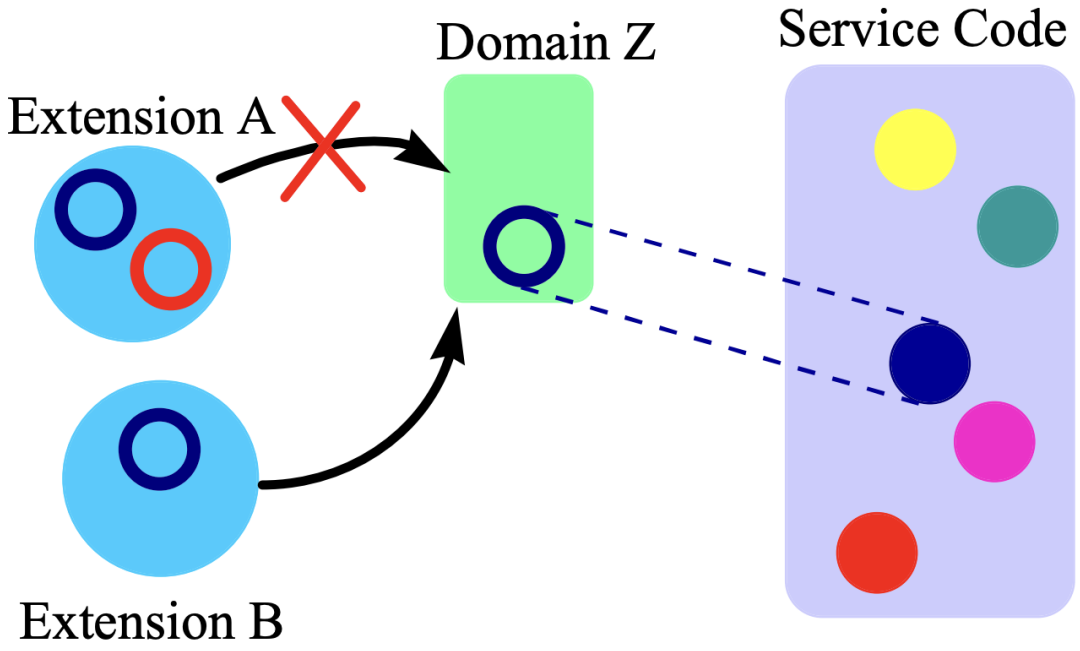
SPIN: performance & safety

- Local protection domains
  - Namespaces for extensions
  - Avoid context switch through a function-call that decides which interfaces (extensions) can be accessed
- Dynamic call binding
  - Binds events to extensions
  - Handler pattern through function call

SPIN: dynamic binding

Goal: control access to interfaces cheaply

Strategy: restrict access at dynamic link-time

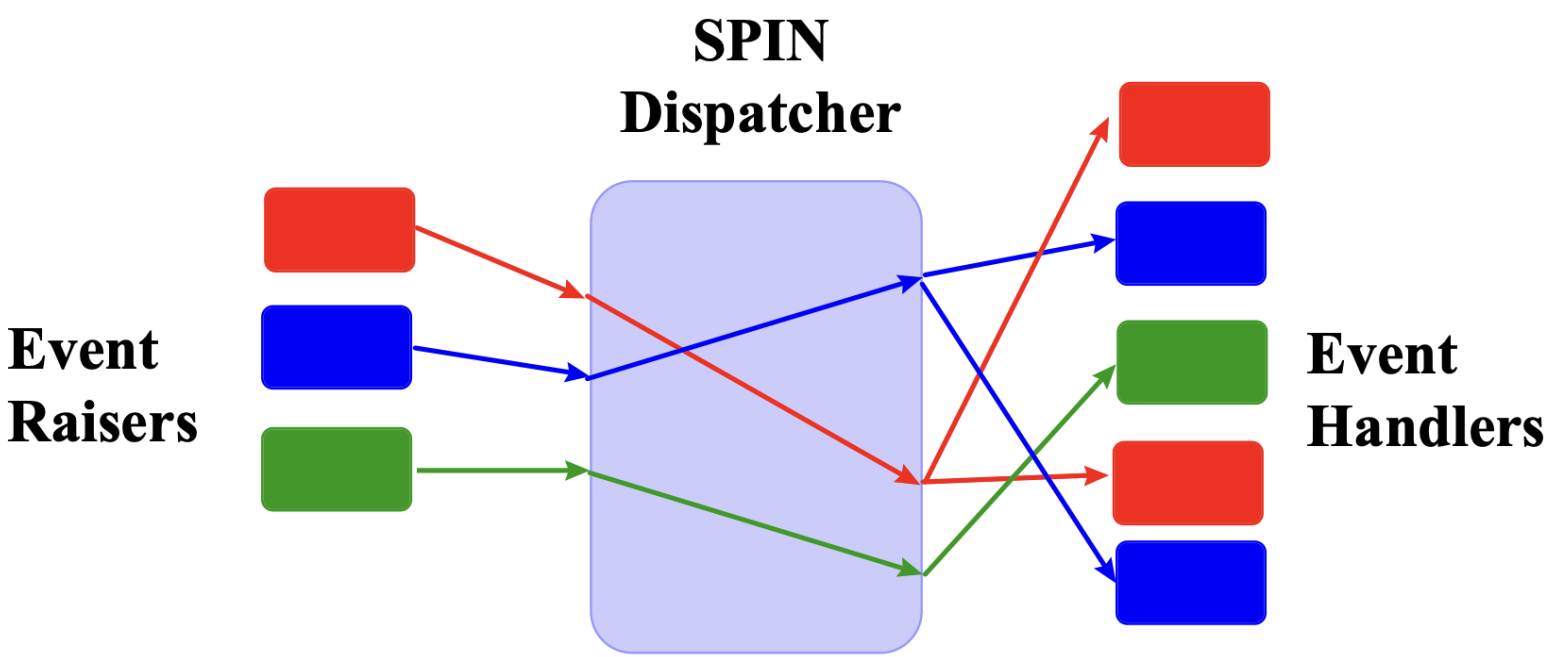


SPIN: extensibility

- Event/handler abstraction
  - Event: a message that announces a change in the state of the system or a request for service
  - (Event) Handler: a procedure that receives the message
- An extension installs a handler on an event by explicitly registering the handler with the event through a central dispatcher
  - Essentially a callback mechanism

SPIN: event dispatcher

Event-based communication model



SPIN: examples

- Memory
  - The kernel controls allocation of physical and virtual addresses capabilities
  - Extensions
    - » Event: page fault
    - » App provides handle for page faults

SPIN: examples

- CPU sharing
  - Based on Modula-3 threads
  - Scheduler multiplexes processor among competing strands
    - » A strand is similar to a thread in traditional operating but no kernel state
    - » Use preemptive round-robin to schedule strands

SPIN: examples

- Network

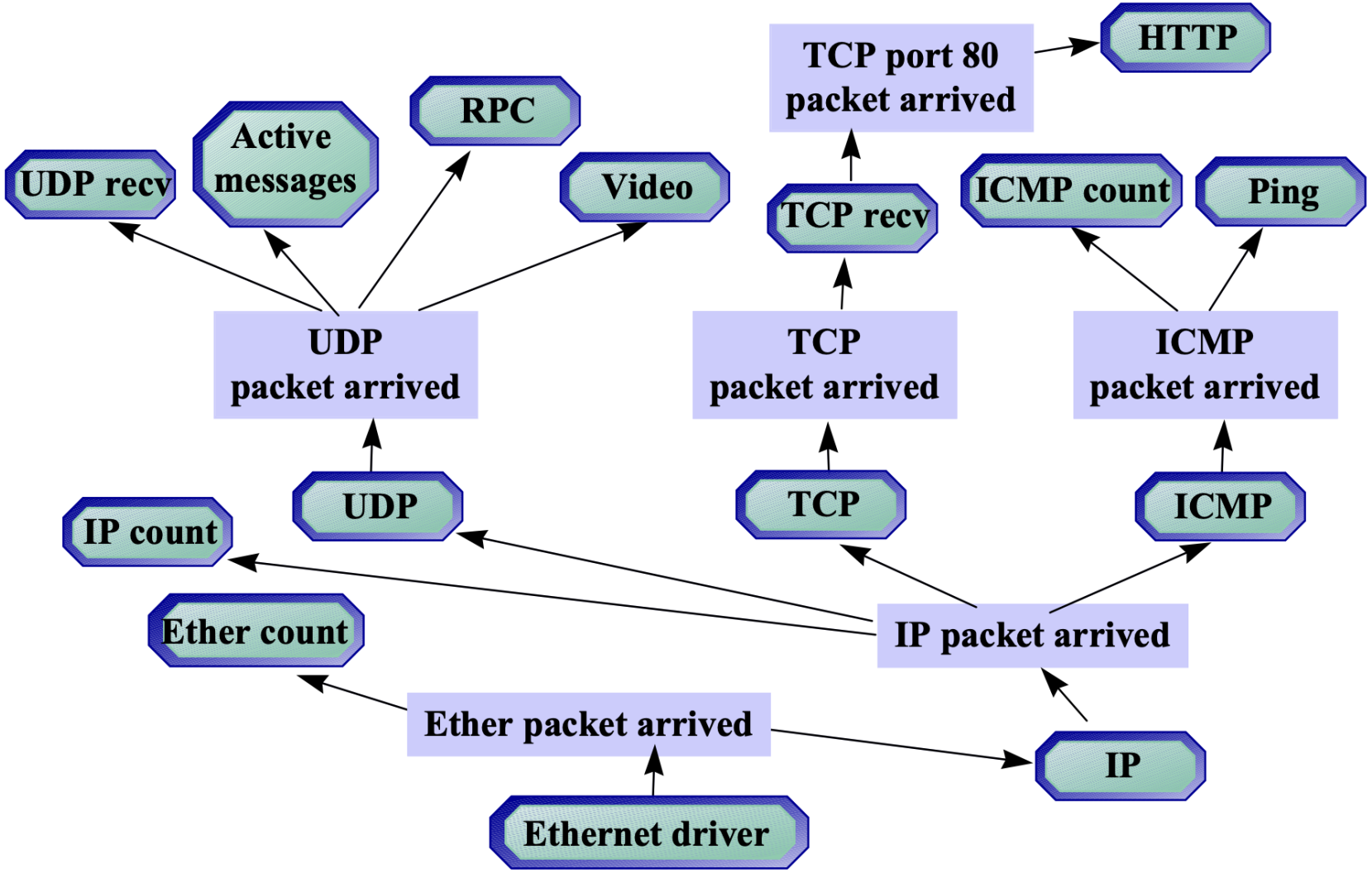
```
INTERFACE Network;
PROCEDURE PacketArrived(p:Pkt);

END Network.
```

Event definition

```
MODULE EthernetDriver;
PROCEDURE Interrupt(p: Pkt) =
  BEGIN
      Network.PacketArrived(p);
  END Interrupt;
```

Event raise

SPIN: protocol graph

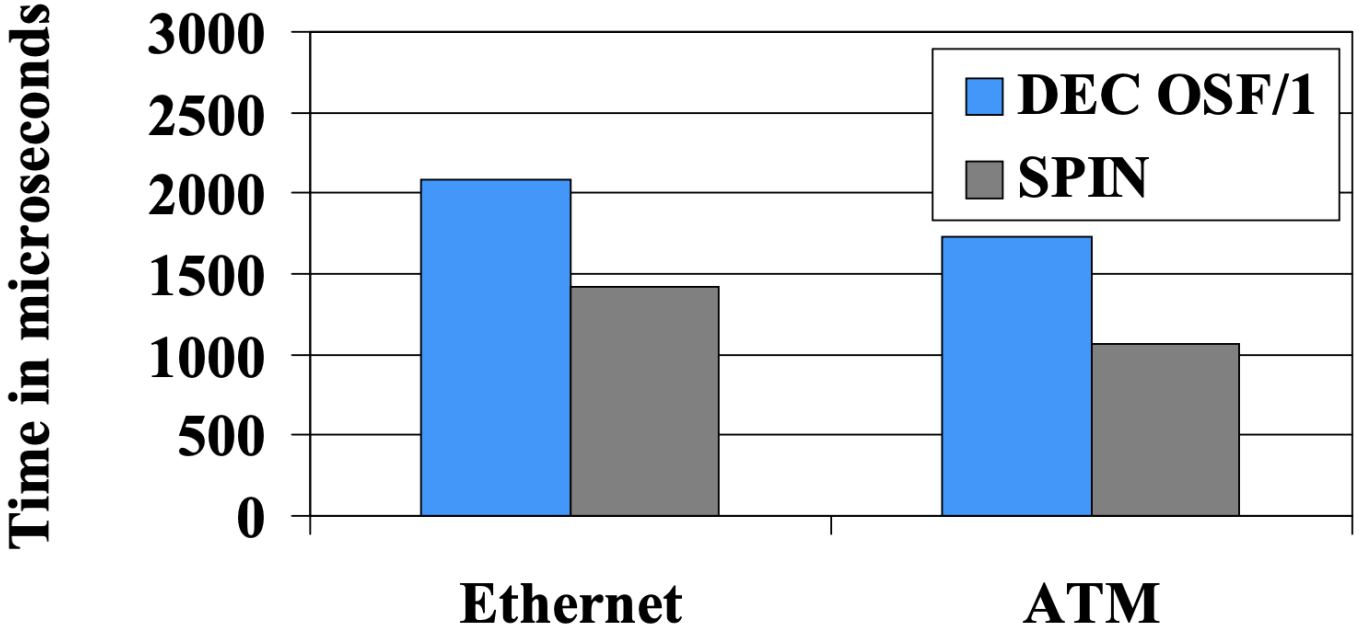


SPIN: other services

- Almost all “system” services are extensions
  - Network protocols
  - File systems
  - System call interface
- SPIN only implements services which cannot be safely implemented as extensions
  - Processor execution state
  - Basic interface to MMU and physical memory
  - Device IO/DMA
  - Dynamic linker and Dispatcher

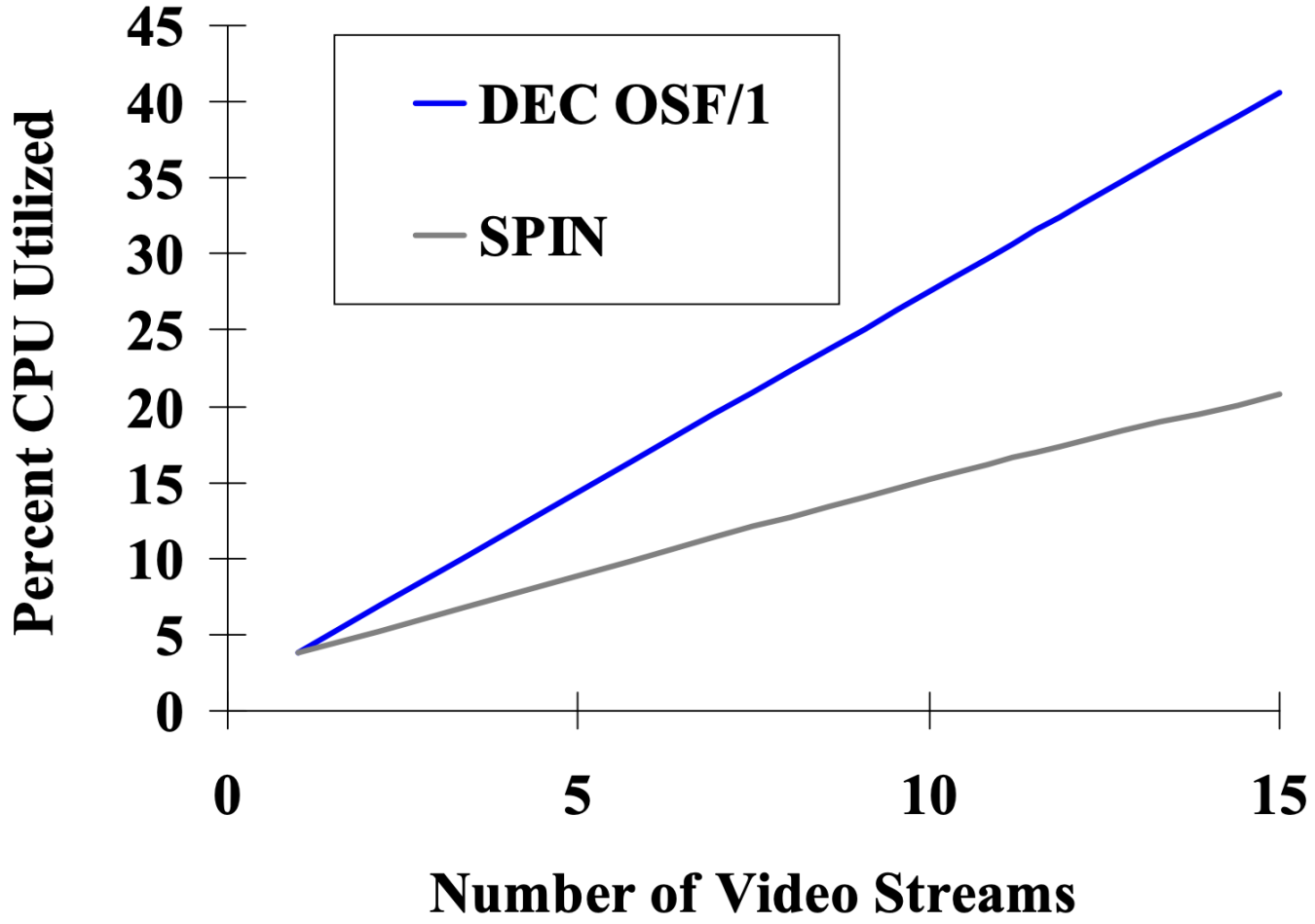
SPIN: performance

- Per-port TCP packet forwarding



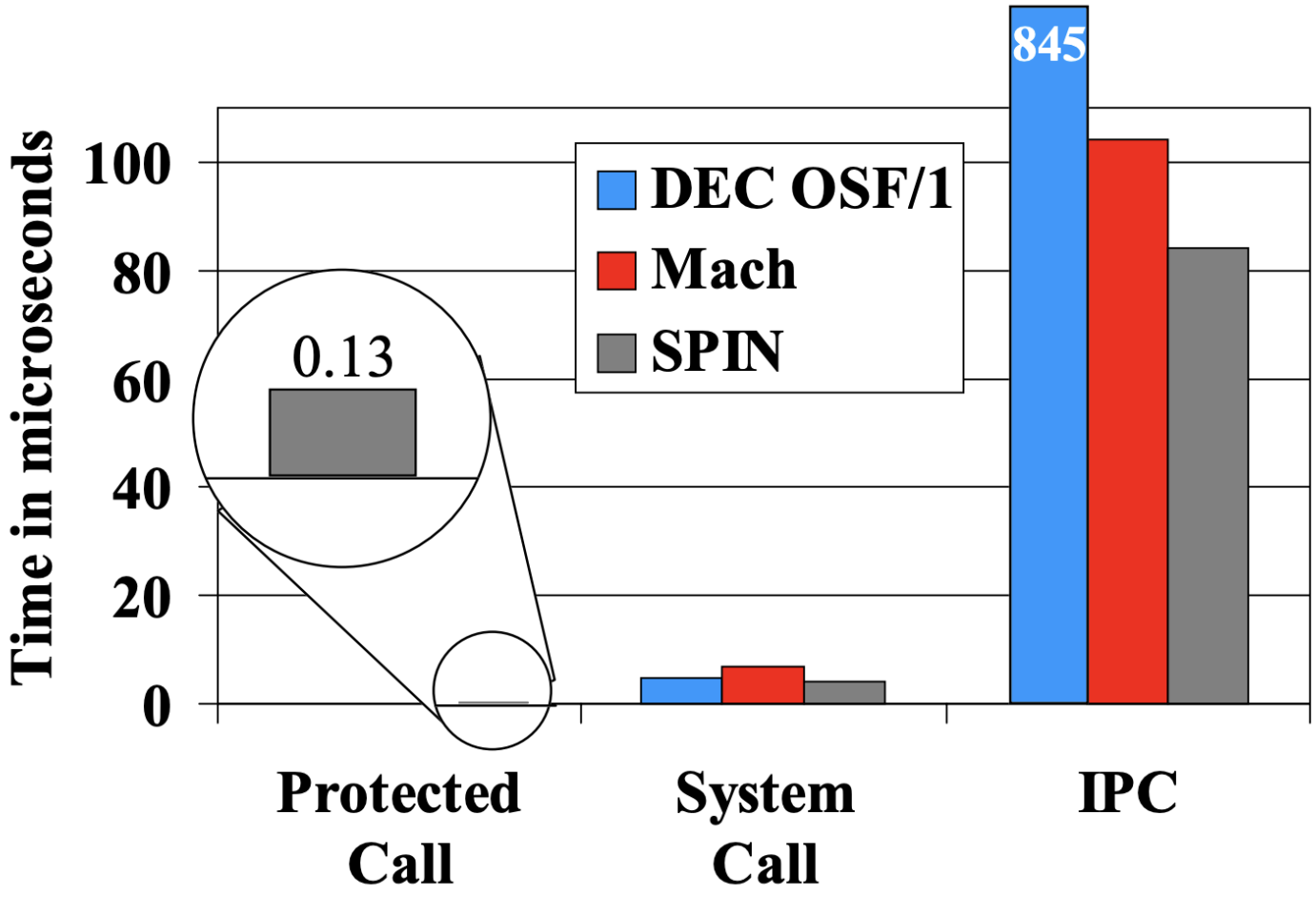
SPIN: performance

- Video service



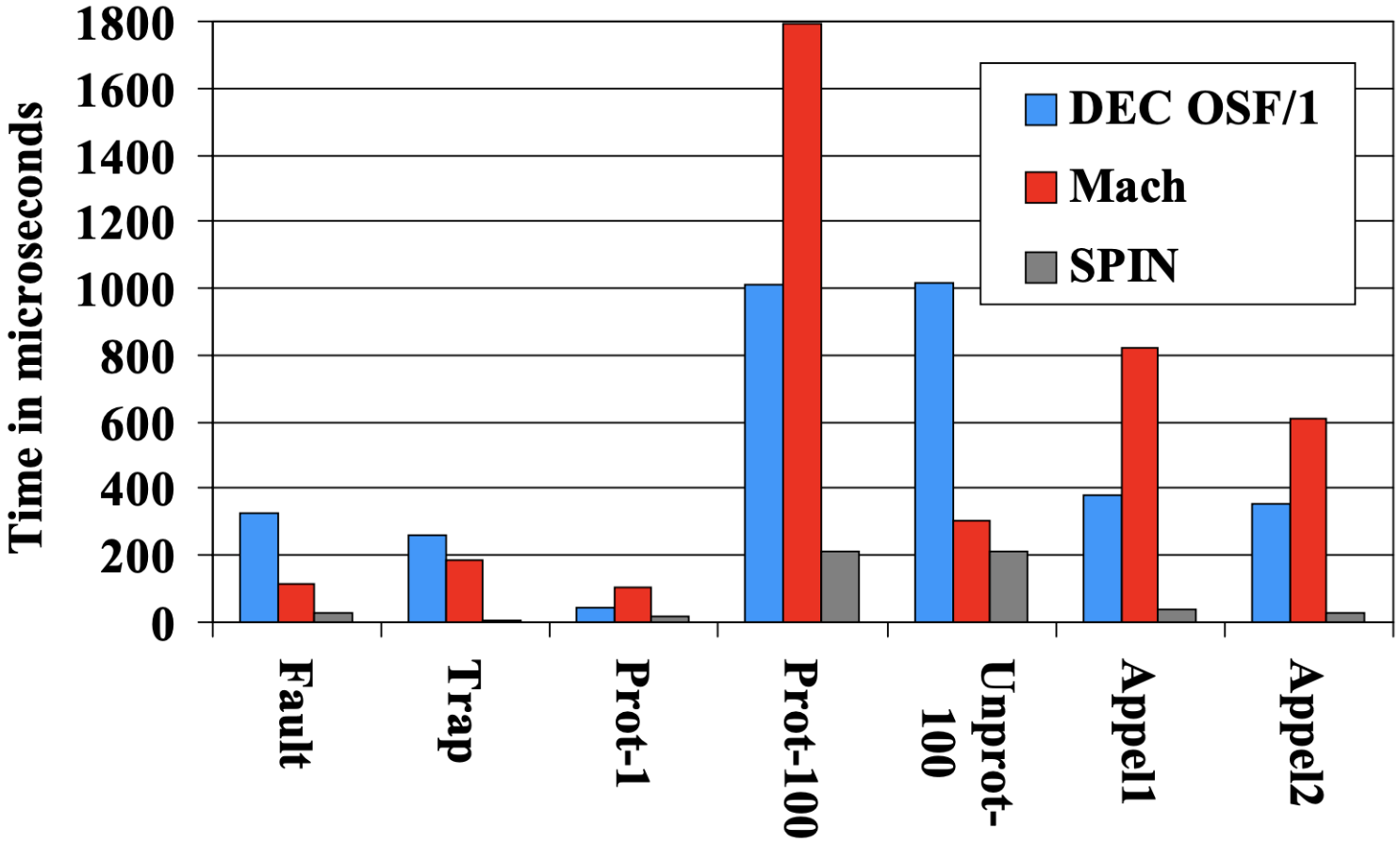
SPIN: performance

- Protected communication



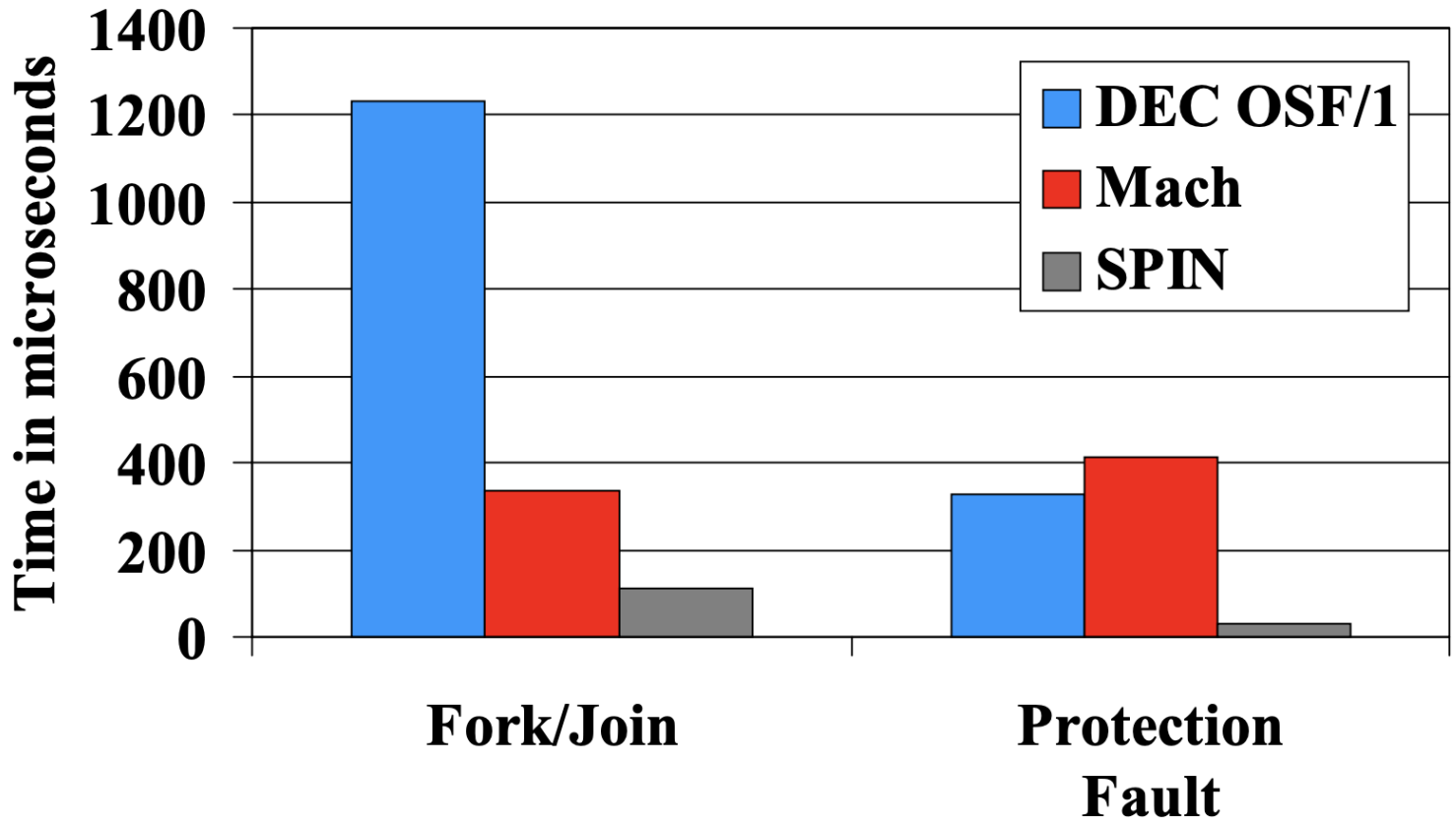
SPIN: performance

- Memory management service



SPIN: performance

- Other services



SPIN: relevance

- Rust for Linux
  - Rust for Linux is an ongoing project to integrate the Rust programming language into the Linux kernel, which has historically been written in C and assembly. The initiative aims to leverage Rust's memory safety to reduce bugs, especially in kernel drivers.
  - Officially supported from Linux 7.0
- Rust in Windows
  - Microsoft is actively integrating Rust into the Windows kernel, marking a major shift toward improved security and memory safety. The first Rust code appeared in the Windows 11 Insider Preview builds in May 2023, with a new Graphics Device Interface (GDI) region module written in Rust—identified as win32kbase_rs.sys.